

# Stellar Spectral Classification with a Multi-Layer Perceptron

Solution Guide

Machine Learning for Optics

2026

## Abstract

This guide walks you through every coding task in the exercise notebook `stellar_spectral_classification_ex` from implementing the Planck function all the way to gradient-based feature importance. Each section maps to a numbered task in the notebook and gives the complete solution code together with an explanation of *why* it works.

## Contents

|          |                                                                    |          |
|----------|--------------------------------------------------------------------|----------|
| <b>1</b> | <b>Overview of the Notebook</b>                                    | <b>3</b> |
| <b>2</b> | <b>Quick Reference: Key PyTorch Patterns</b>                       | <b>3</b> |
| 2.1      | Building a network with <code>nn.Sequential</code> . . . . .       | 3        |
| 2.2      | The training step . . . . .                                        | 3        |
| 2.3      | <code>model.train()</code> vs. <code>model.eval()</code> . . . . . | 3        |
| 2.4      | Disabling gradients with <code>torch.no_grad()</code> . . . . .    | 4        |
| <b>3</b> | <b>Task 1.1: Implement Planck's Law</b>                            | <b>4</b> |
| 3.1      | What to do . . . . .                                               | 4        |
| 3.2      | Why <code>np.exp(1)</code> ? . . . . .                             | 4        |
| 3.3      | Plotting the curves . . . . .                                      | 4        |
| <b>4</b> | <b>Task 3.1–3.2: Load and Visualise the Dataset</b>                | <b>5</b> |
| 4.1      | Loading the <code>.npz</code> file . . . . .                       | 5        |
| 4.2      | Task 3.2: Visualising spectra . . . . .                            | 5        |
| <b>5</b> | <b>Task 4.1: Standardise and Split</b>                             | <b>5</b> |
| 5.1      | Per-spectrum standardisation . . . . .                             | 5        |
| 5.2      | Train / validation / test split . . . . .                          | 6        |
| <b>6</b> | <b>Task 5.1: Define the MLP</b>                                    | <b>6</b> |
| 6.1      | How the loop works . . . . .                                       | 7        |
| 6.2      | Verifying the model . . . . .                                      | 7        |
| <b>7</b> | <b>Task 6.1: Training Loop</b>                                     | <b>7</b> |
| 7.1      | The <code>run_epoch</code> function . . . . .                      | 7        |
| 7.2      | The outer training loop . . . . .                                  | 8        |
| 7.3      | Task 6.2: Plot training curves . . . . .                           | 9        |
| <b>8</b> | <b>Task 7: Evaluation</b>                                          | <b>9</b> |
| 8.1      | Task 7.1: Test accuracy and classification report . . . . .        | 9        |
| 8.2      | Task 7.2: Confusion matrix . . . . .                               | 10       |

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>9 Task 8.1: Gradient-Based Feature Importance</b> | <b>10</b> |
| 9.1 The mathematics . . . . .                        | 10        |
| 9.2 Implementation . . . . .                         | 11        |
| 9.3 Plotting . . . . .                               | 11        |
| <b>10 Common Mistakes Checklist</b>                  | <b>12</b> |
| <b>11 Bonus Exercises: Hints</b>                     | <b>12</b> |
| 11.1 Exercise A: Architecture search . . . . .       | 12        |
| 11.2 Exercise B: Noise robustness . . . . .          | 12        |
| 11.3 Exercise C: PCA . . . . .                       | 12        |

## 1 Overview of the Notebook

The exercise notebook contains eight sections. The table below shows each section, the task(s) inside it, and what you need to produce.

| Sec. | Task | What to implement                                          |
|------|------|------------------------------------------------------------|
| 1    | 1.1  | <code>planck()</code> function + plot normalised curves    |
| 3    | 3.1  | Load <code>stellar_spectra.npz</code> and inspect contents |
|      | 3.2  | Visualise spectra by class                                 |
| 4    | 4.1  | Standardise spectra, split into train/val/test             |
| 5    | 5.1  | Define the <code>StellarMLP</code> class                   |
| 6    | 6.1  | Training loop (forward, backward, scheduler)               |
|      | 6.2  | Plot training curves                                       |
| 7    | 7.1  | Evaluate on the test set                                   |
|      | 7.2  | Plot the confusion matrix                                  |
| 8    | 8.1  | Gradient-based feature importance                          |

Table 1: Tasks in the exercise notebook.

## 2 Quick Reference: Key PyTorch Patterns

Before diving into the tasks, here are the PyTorch patterns you will use throughout the notebook.

### 2.1 Building a network with `nn.Sequential`

```

1 net = nn.Sequential(
2     nn.Linear(in_features, hidden),
3     nn.BatchNorm1d(hidden),
4     nn.ReLU(),
5     nn.Dropout(p),
6     nn.Linear(hidden, out_features),
7 )

```

`nn.Sequential` chains layers so that the output of each feeds into the next. You call the whole block as `net(x)`.

### 2.2 The training step

Every training iteration follows the same four lines:

```

1 optimiser.zero_grad()      # reset gradients from previous step
2 loss.backward()            # compute gradients via backpropagation
3 optimiser.step()           # update weights

```

#### Order matters

`zero_grad()` must come **before** `backward()`. If you forget it, gradients accumulate across batches and training diverges.

### 2.3 `model.train()` vs. `model.eval()`

Two layers behave differently at training vs. inference time:

- **BatchNorm**: uses mini-batch statistics during training, running averages during evaluation.
- **Dropout**: drops neurons during training, uses all neurons during evaluation.

Always call `model.eval()` before validation or test loops, and `model.train()` before training loops.

## 2.4 Disabling gradients with `torch.no_grad()`

During evaluation you do not need gradients. Wrapping the loop in `with torch.no_grad():` saves memory and speeds things up.

# 3 Task 1.1: Implement Planck's Law

## 3.1 What to do

The function `planck(lam, T)` should return the spectral radiance  $B_\lambda(\lambda, T)$ . The formula is given in the notebook:

$$B_\lambda = \frac{2hc^2}{\lambda^5} \frac{1}{\exp(hc/\lambda k_B T) - 1}$$

### Solution code

```
1 def planck(lam, T):
2     lam = np.asarray(lam, dtype=np.float64)
3     return (2*h*c**2 / lam**5) / np.expm1(h*c / (lam*kB*T))
```

## 3.2 Why `np.expm1`?

`np.expm1(x)` computes  $e^x - 1$  in a way that is accurate even when  $x$  is very small (where naive `np.exp(x) - 1` loses precision due to floating-point cancellation). For hot stars where  $hc/\lambda k_B T$  is small, this matters.

## 3.3 Plotting the curves

For each temperature, compute `B = planck(lam_m, T)`, normalise by dividing by `B.max()`, and plot against the wavelength grid. Mark the visible band with `ax.axvspan(380, 700, alpha=0.1)`.

```
1 for label, T in temperatures.items():
2     B = planck(lam_m, T)
3     ax.plot(lam_nm, B / B.max(), label=label)
4 ax.axvspan(380, 700, alpha=0.1, color="yellow", label="visible")
5 ax.set_xlabel("Wavelength (nm)")
6 ax.set_ylabel("Normalised radiance")
7 ax.legend(fontsize=8); ax.set_title("Planck curves")
```

## 4 Task 3.1–3.2: Load and Visualise the Dataset

### 4.1 Loading the .npz file

#### Solution code

```

1 data = np.load("stellar_spectra.npz", allow_pickle=True)
2
3 spectra      = data["spectra"]          # (3500, 1040)
4 y_all        = data["labels"]           # (3500,)
5 wavelengths   = data["wavelengths"]     # (1040,)
6 CLASS_NAMES  = data["class_names"]     # ['O', 'B', 'A', 'F', 'G', 'K', 'M',
7     'J']
8 temps        = data["temperatures"]    # (3500,)
9
10 NUM_FEATURES = spectra.shape[1]        # 1040
11 NUM_CLASSES  = len(CLASS_NAMES)        # 7
12
13 print(f"Spectra: {spectra.shape}")
14 print(f"Classes: {list(CLASS_NAMES)}")
15 for i, name in enumerate(CLASS_NAMES):
16     print(f" {name}: {(y_all == i).sum()} spectra")

```

#### allow\_pickle=True

The `class_names` array contains Python strings, which NumPy stores as pickled objects. Without this flag, `np.load` will refuse to load it.

### 4.2 Task 3.2: Visualising spectra

Plot 3–5 spectra from each class on the same axes. Use the wavelength grid as the x-axis:

```

1 fig, axes = plt.subplots(4, 2, figsize=(13, 12),
2     sharex=True, sharey=True)
3 for i, (ax, name) in enumerate(
4     zip(axes.flat, CLASS_NAMES)):
5     mask = y_all == i
6     for j in range(min(5, mask.sum())):
7         ax.plot(wavelengths, spectra[mask][j], alpha=0.6)
8     ax.set_title(f"Type {name}")
9     ax.set_ylabel("Flux")
10 axes[-1, 0].set_xlabel("Wavelength (nm)")
11 axes[-1, 1].set_xlabel("Wavelength (nm)")
12 if len(CLASS_NAMES) < len(axes.flat):
13     axes[-1, -1].axis("off")
14 plt.tight_layout(); plt.show()

```

**What to notice:** hot types (O, B) peak in the blue/UV; cool types (K, M) peak in the red/infrared. Absorption features (dips) appear at different wavelengths for different types.

## 5 Task 4.1: Standardise and Split

### 5.1 Per-spectrum standardisation

Each spectrum is a row of 1040 flux values. Compute the mean and standard deviation *along the wavelength axis* (axis 1):

**Solution code**

```
1 mu = spectra.mean(axis=1, keepdims=True)
2 std = spectra.std(axis=1, keepdims=True)
3 X_norm = (spectra - mu) / (std + 1e-8)
```

**Why per spectrum**

Per-feature standardisation (subtracting the mean flux at each wavelength across all spectra) would remove the continuum shape differences between types. Per-spectrum standardisation preserves relative absorption depths while centering each spectrum individually.

## 5.2 Train / validation / test split

Use `random_split` from PyTorch:

```
1 X_tensor = torch.tensor(X_norm, dtype=torch.float32)
2 y_tensor = torch.tensor(y_all, dtype=torch.long)
3 dataset = TensorDataset(X_tensor, y_tensor)
4
5 n_total = len(dataset)
6 n_train = int(0.70 * n_total)
7 n_val = int(0.15 * n_total)
8 n_test = n_total - n_train - n_val
9
10 train_ds, val_ds, test_ds = random_split(
11     dataset, [n_train, n_val, n_test],
12     generator=torch.Generator().manual_seed(SEED)
13 )
```

**Common pitfall: wrong dtype**

Labels must be `torch.long` (64-bit integers). `CrossEntropyLoss` will raise an error if you pass `float32` labels. Spectra should be `float32`.

## 6 Task 5.1: Define the MLP

The architecture is three hidden layers (256, 128, 64) with BatchNorm, ReLU, and Dropout after each, plus a final linear output layer.

## Solution code

```

1 class StellarMLP(nn.Module):
2     def __init__(self, n_features, n_classes,
3                   hidden=(256, 128, 64), drop=0.3):
4         super().__init__()
5         layers = []
6         prev = n_features
7         for h in hidden:
8             layers += [
9                 nn.Linear(prev, h),
10                nn.BatchNorm1d(h),
11                nn.ReLU(),
12                nn.Dropout(drop),
13            ]
14            prev = h
15        layers.append(nn.Linear(prev, n_classes))
16        self.net = nn.Sequential(*layers)
17
18    def forward(self, x):
19        return self.net(x)

```

## 6.1 How the loop works

The list `layers` is built iteratively. For each hidden size `h` in (256, 128, 64):

1. `Linear(prev, h)` projects from the previous size to `h`.
2. `BatchNorm1d(h)` normalises across the batch.
3. `ReLU()` applies the non-linearity.
4. `Dropout(drop)` regularises by randomly zeroing activations.

After the loop, one final `Linear(64, 7)` maps to the 7 class logits. The `*` in `nn.Sequential(*layers)` unpacks the list into positional arguments.

## 6.2 Verifying the model

After creating the model, print it and check the parameter count:

```

1 model = StellarMLP(NUM_FEATURES, NUM_CLASSES).to(device)
2 total = sum(p.numel() for p in model.parameters()
3             if p.requires_grad)
4 print(f"Trainable parameters: {total:,}")
5 # Expected: ~309,000

```

# 7 Task 6.1: Training Loop

## 7.1 The run\_epoch function

This function runs one pass through a `DataLoader`. The key difference between training and evaluation is whether gradients are computed.

## Solution code for the inner loop

```

1 def run_epoch(loader, train=True):
2     model.train(train)
3     total_loss, correct, total = 0.0, 0, 0
4     ctx = torch.enable_grad() if train else torch.no_grad()
5     with ctx:
6         for xb, yb in loader:
7             xb, yb = xb.to(device), yb.to(device)
8             logits = model(xb)
9             loss = criterion(logits, yb)
10            if train:
11                optimiser.zero_grad()
12                loss.backward()
13                optimiser.step()
14                total_loss += loss.item() * len(yb)
15                correct += (logits.argmax(1) == yb).sum().item()
16                total += len(yb)
17    return total_loss / total, correct / total

```

## Line by line:

1. `model.train(train)` sets training or eval mode.
2. `logits = model(xb)` is the forward pass.
3. `loss = criterion(logits, yb)` computes cross-entropy.
4. `optimiser.zero_grad()` clears old gradients.
5. `loss.backward()` computes new gradients.
6. `optimiser.step()` updates weights.
7. `logits.argmax(1)` picks the class with the highest score; comparing to `yb` counts correct predictions.
8. We multiply `loss.item()` by batch size because PyTorch's `CrossEntropyLoss` returns the mean over the batch by default.

## 7.2 The outer training loop

## Solution code

```

1 for epoch in range(1, EPOCHS + 1):
2     tl, ta = run_epoch(train_dl, train=True)
3     vl, va = run_epoch(val_dl, train=False)
4     scheduler.step(vl)
5     history["train_loss"].append(tl)
6     history["val_loss"].append(vl)
7     history["train_acc"].append(ta)
8     history["val_acc"].append(va)
9     if epoch % 10 == 0:
10        lr = optimiser.param_groups[0]["lr"]
11        print(f"epoch {epoch:3d} | "
12              f"train {tl:.4f}/{ta:.3f} | "
13              f"val {vl:.4f}/{va:.3f} | "
14              f"lr {lr:.1e}")

```

**ReduceLROnPlateau**

`scheduler.step(v1)` passes the validation loss to the scheduler. If the loss has not improved for 5 consecutive epochs (`patience=5`), the learning rate is multiplied by 0.5 (`factor=0.5`). This lets the model take smaller steps when it stops making progress.

**7.3 Task 6.2: Plot training curves**

```

1 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
2 epochs = range(1, EPOCHS + 1)
3
4 ax1.plot(epochs, history["train_loss"], label="train")
5 ax1.plot(epochs, history["val_loss"], label="val")
6 ax1.set_xlabel("Epoch"); ax1.set_ylabel("Loss")
7 ax1.legend(); ax1.set_title("Loss")
8
9 ax2.plot(epochs, history["train_acc"], label="train")
10 ax2.plot(epochs, history["val_acc"], label="val")
11 ax2.set_xlabel("Epoch"); ax2.set_ylabel("Accuracy")
12 ax2.legend(); ax2.set_title("Accuracy")
13
14 plt.tight_layout(); plt.show()

```

**What to look for:** Train and val curves should converge together. If the train loss keeps dropping while val loss flattens or rises, you are overfitting. Dropout and early stopping help with this.

**8 Task 7: Evaluation****8.1 Task 7.1: Test accuracy and classification report****Solution code**

```

1 model.eval()
2 all_preds, all_labels = [], []
3 with torch.no_grad():
4     for xb, yb in test_dl:
5         xb = xb.to(device)
6         preds = model(xb).argmax(1).cpu()
7         all_preds.append(preds)
8         all_labels.append(yb)
9
10 all_preds = torch.cat(all_preds).numpy()
11 all_labels = torch.cat(all_labels).numpy()

```

The key pattern: loop over batches, move input to device, get predictions with `.argmax(1)`, move back to CPU, collect into lists, then concatenate at the end.

**Expected accuracy:** > 95% with 500 samples per class. Types B/O and F/G are the hardest to separate because their temperature ranges overlap.

## 8.2 Task 7.2: Confusion matrix

### Solution code

```

1  cm = confusion_matrix(all_labels, all_preds)
2
3  fig, ax = plt.subplots(figsize=(7, 6))
4  im = ax.imshow(cm, cmap="Blues")
5  ax.set_xticks(range(NUM_CLASSES))
6  ax.set_xticklabels(list(CLASS_NAMES))
7  ax.set_yticks(range(NUM_CLASSES))
8  ax.set_yticklabels(list(CLASS_NAMES))
9  ax.set_xlabel("Predicted"); ax.set_ylabel("True")
10 ax.set_title("Confusion Matrix")
11
12 for i in range(NUM_CLASSES):
13     for j in range(NUM_CLASSES):
14         val = cm[i, j]
15         colour = "white" if val > cm.max()/2 else "black"
16         ax.text(j, i, str(val), ha="center",
17                va="center", color=colour, fontsize=10)
18
19 plt.colorbar(im, ax=ax, shrink=0.8)
20 plt.tight_layout(); plt.show()

```

The double loop adds numeric annotations to each cell. White text is used on dark cells and black on light cells for readability.

## 9 Task 8.1: Gradient-Based Feature Importance

This is the most advanced task. The idea: compute how much the predicted probability for the *correct* class changes when each input wavelength changes slightly. Wavelengths with large gradients are the ones the network relies on for its decision.

### 9.1 The mathematics

For each test spectrum  $\mathbf{x}_n$  with true label  $y_n$ , compute:

$$I_j = \frac{1}{N} \sum_{n=1}^N \left| \frac{\partial p_{y_n}(\mathbf{x}_n)}{\partial x_{n,j}} \right|$$

where  $p_{y_n}$  is the softmax probability for the correct class and  $x_{n,j}$  is the flux at wavelength  $j$ .

## 9.2 Implementation

### Solution code

```

1 model.eval()
2 importance = torch.zeros(NUM_FEATURES)
3 n_samples = 0
4
5 for xb, yb in test_dl:
6     xb = xb.to(device).requires_grad_(True)
7     logits = model(xb)
8     probs = F.softmax(logits, dim=1)
9     # Select the probability of the true class
10    true_probs = probs.gather(1, yb.to(device))
11                        .unsqueeze(1).squeeze(1)
12    # Sum so we can call backward (not mean, to get
13    # per-sample gradients summed)
14    true_probs.sum().backward()
15    importance += xb.grad.abs().sum(0).cpu()
16    n_samples += len(yb)
17
18 importance /= n_samples

```

#### Key details:

1. `requires_grad_(True)` on the input tensor enables gradient computation with respect to the input (normally only parameters have gradients).
2. `probs.gather(1, yb...)` selects the softmax probability of the true class for each sample in the batch.
3. `.sum().backward()` computes gradients for all samples in the batch simultaneously.
4. `xb.grad.abs().sum(0)` sums the absolute gradients across the batch dimension.

## 9.3 Plotting

```

1 fig, ax = plt.subplots(figsize=(12, 4))
2 ax.plot(wavelengths, importance.numpy(), color="darkblue",
3         linewidth=0.8)
4 ax.set_xlabel("Wavelength (nm)")
5 ax.set_ylabel("Mean |gradient|")
6 ax.set_title("Gradient-based Feature Importance")
7
8 # Mark known absorption lines
9 lines = {"H-alpha": 656.3, "H-beta": 486.1,
10         "Na D": 589.3, "Ca II K": 393.4,
11         "He I": 587.6, "TiO": 712.0}
12 for name, wl in lines.items():
13     ax.axvline(wl, color="red", alpha=0.4, ls="--")
14     ax.text(wl+2, ax.get_ylim()[1]*0.9, name,
15           fontsize=7, rotation=90, va="top")
16
17 plt.tight_layout(); plt.show()

```

**What to verify:** the importance peaks should align with the known absorption lines (H-alpha at 656.3nm, CaII at 393–397nm, etc.). This confirms that the network has learned physically meaningful features rather than memorising noise.

## 10 Common Mistakes Checklist

### Before you submit

1. **Wrong dtype for labels.** Labels must be `torch.long`, not `float32`.
2. **Forgot `model.eval()`.** BatchNorm and Dropout behave differently at test time.
3. **Forgot `optimiser.zero_grad()`.** Without it, gradients accumulate from previous batches.
4. **Passed softmax to CrossEntropyLoss.** It expects raw logits; it applies log-softmax internally.
5. **Forgot `.to(device)`.** Both model and data must be on the same device (CPU or GPU).
6. **Standardised per-feature instead of per-spectrum.** Per-feature standardisation removes the continuum shape, making types harder to distinguish.
7. **Feature importance without `requires_grad_(True)`.** Input tensors do not track gradients by default.
8. **Not using `torch.no_grad()` during evaluation.** It works without it, but wastes memory and is slower.
9. **Confusing random\_split sizes.** The sizes must sum exactly to `len(dataset)`. Use `n_test = n_total - n_train - n_val` to avoid off-by-one errors.

## 11 Bonus Exercises: Hints

### 11.1 Exercise A: Architecture search

Loop over different hidden configurations:

```
1 configs = [(128,), (256, 128), (256, 128, 64),
2             (512, 256, 128, 64)]
3 for hidden in configs:
4     model = StellarMLP(NUM_FEATURES, NUM_CLASSES,
5                       hidden=hidden).to(device)
6     # train and record accuracy...
```

### 11.2 Exercise B: Noise robustness

Add Gaussian noise to the already-standardised test spectra:

```
1 noise_std = 0.5 # try 0.1, 0.5, 1.0, 2.0
2 for xb, yb in test_dl:
3     xb_noisy = xb + noise_std * torch.randn_like(xb)
4     # evaluate on xb_noisy...
```

### 11.3 Exercise C: PCA

```
1 from sklearn.decomposition import PCA
2
3 # Fit on training data only
4 train_idx = train_ds.indices
5 pca = PCA(n_components=0.99) # keep 99% variance
6 X_pca = pca.fit_transform(X_norm[train_idx])
7 print(f"Components: {pca.n_components_}") # typically ~20-30
8
9 # Transform val/test using the same PCA
```

```
10 X_val_pca = pca.transform(X_norm[val_ds.indices])
11 X_test_pca = pca.transform(X_norm[test_ds.indices])
```

## References

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. <https://www.deeplearningbook.org>
- PyTorch documentation: <https://pytorch.org/docs/stable/>
- PyTorch tutorials: <https://pytorch.org/tutorials/beginner/basics/intro.html>